

Smart MCQ Solver Challenge

Deep Learning & Generative AI Project Report

Roll Number: 24f2003563 | Term: T2-2026 | W&B Project: 24f2003563-t22026

1. Abstract / Executive Summary

This project has the problem statement the Smart MCQ Solver Challenge on Kaggle: given a question contains a prompt along with five possible answer options labeled (A-E), predict the top three most correct answers. The objective is to predict the top three most likely correct answers in ranked order. Submissions will be evaluated using Mean Average Precision at 3 (MAP@3), where models receive higher scores when the correct answer appears earlier in their ranked predictions. Three model were built and compared: (1) a fine-tuned transformer (DeBERTa-v3-large + LoRA), (2) a custom Text-CNN, and (3) an LSTM, both built from scratch. All experiments were tracked in Weights & Biases. On the held-out validation split, the from-scratch Text-CNN achieved the strongest individual score (MAP@3 = 0.972, Accuracy = 0.980, F1 = 0.950), ahead of both the fine-tuned transformer (MAP@3 = 0.934) and the LSTM (MAP@3 = 0.921). A weighted average ensemble of all three models (LoRA 0.65 / Text-CNN 0.20 / LSTM 0.15) improved on every individual model, reaching MAP@3 = 0.993, Accuracy = 0.995, and F1 = 0.987 on validation, and was used to generate the final competition submission.

2. Introduction

2.1 Problem Statement

Each question contains a prompt along with five possible answer options labeled (A-E), predict the top three most correct answers. The objective is to predict the top three most likely correct answers in ranked order. Submissions will be evaluated using Mean Average Precision at 3 (MAP@3), where models receive higher scores when the correct answer appears earlier in their ranked predictions a score of 1, 1/2, or 1/3 is awarded if the correct answer appears first, second, or third respectively, and 0 otherwise; scores are averaged across all test questions.

2.2 Project Objective

The goal was to build and compare three model on the same binary-classification task - a fine-tuned pretrained transformer (DeBERTa-v3-large + LoRA), a custom convolutional model built from scratch (Text-CNN), and a recurrent model built from scratch (LSTM) - and then combine their predictions in a weighted ensemble to maximise MAP@3. A secondary objective was to gain hands-on experience with the Hugging Face / PyTorch / PEFT stack and with tracking via Weights & Biases.

2.3 Report Structure

Section 3 covers the dataset and preprocessing, including the exploratory analysis used to characterise the data. Section 4 describes the tokenization strategy used for each family of models. Section 5 details the architecture, design, and training configuration of each of the three models. Section 6 presents training curves, a comparative table across all models and the ensemble, and the evaluation metrics used. Section 7 reflects on key learnings, challenges encountered, and directions for future work. Section 8 has references resources that cited or used in project.

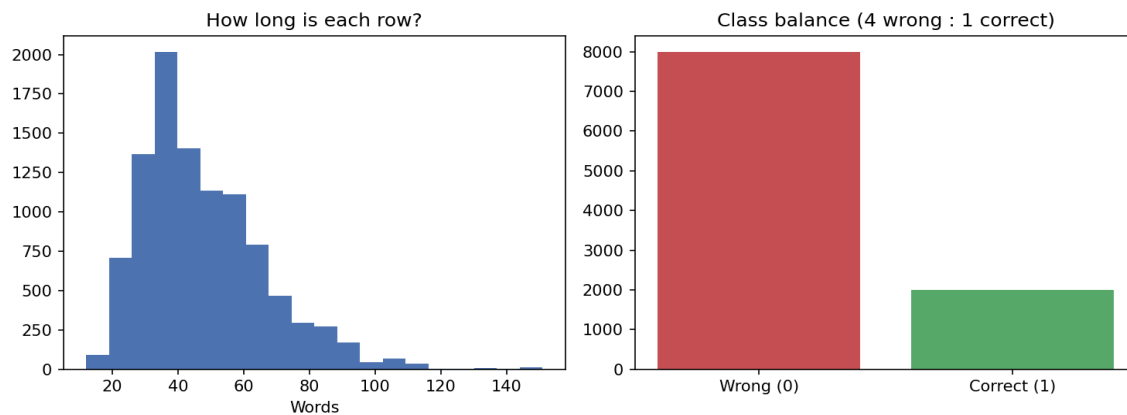
3. Dataset & Preprocessing

3.1 Dataset Description

The raw training set contains 2,000 questions (columns: id, prompt, options A-E, and the correct answer label), and the raw test set contains 500 questions (columns - the same fields minus the answer). Each question was exploded into five rows, one per option(A-E), turning the task into binary classification (Is this option correct). This produced 10,000 labelled training rows and 2,500 unlabelled test rows.

3.2 Exploratory Data Analysis

- No missing values were found in any column of the training data.
- Every question has exactly five options (A-E), with exactly one correct - so the expected positive rate per exploded row is 0.20, and the exploded dataset is naturally imbalanced 4 wrong, 1 correct.
- Text length: the average exploded row ("Question: ... Candidate Answer: ...") is 47.4 words, with a maximum of 151 words - short enough that a 64-token window (Models 2-3) and a 256-token window (Model 1) both comfortably train.



Row-length distribution (left) and class balance across the 10,000 exploded training rows (right).

3.3 Data Preprocessing

A custom `build_dataset()` function expands each question into five (prompt, option, label) rows and formats the input text as "Question: {prompt} Candidate Answer: {option}". The train/validation split (85/15) was performed at the question level, not the row level - splitting on the unique question id and only then selecting all five of that question's rows - which keeps all five options of a given question in the same split and avoids leaking partial information about a question across the split boundary.

3.4 Data Augmentation

No data augmentation was applied. The dataset's short, templated text and modest size (10,000 rows) made augmentation a lower priority than getting three clean baseline architectures working correctly; it is noted as a possible future-work direction.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best individual transformer model, DeBERTa-v3-large, uses its native SentencePiece-based subword tokenizer (a BPE-style vocabulary of roughly 128K tokens), loaded directly via Hugging Face's AutoTokenizer for the `microsoft/deberta-v3-large` model.

Justification: Subword tokenization handles rare words, numbers, and terms in the question/option text easily by tokenize smaller sub-word pieces instead of a single <unk> token, and re-using the exact tokenizer the model was pretrained with transfer of its pretrained language understanding to this task.

4.2 Implementation Details

Tokenization used Hugging Face transformers AutoTokenizer with batched .map() calls over a Hugging Face Dataset, fixed-length padding to max_length=256, and truncation enabled as a safety measure (EDA confirmed true maximum row length of 151 words never actually required truncation). Batches were converted into a plain PyTorch Dataset (input_ids, attention_mask, labels tensors) rather than relying on Hugging Face datasets library.

4.3 Experimental Comparison

For the from-scratch Text-CNN and LSTM models, the same DeBERTa-v3 subword tokenizer used for Model 1 was reused rather than building a separate custom vocabulary. Text is passed through the identical AutoTokenizer, truncated and padded to a much shorter max_length=64 and the resulting subword-id sequences feed directly into embedding layer sized to the tokenizer's vocabulary with tokenizer.pad_token_id used as the padding index.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer Model (DeBERTa-v3-large + LoRA)

Architecture: DeBERTa-v3-large (model card: huggingface.co/microsoft/deberta-v3-large) is a 24-layer encoder-only transformer that separately encodes token content and relative position (disentangled attention) rather than adding position embeddings to content embeddings as in BERT/Roberta, and was pretrained with a replaced-token-detection objective rather than standard masked-language-modelling.

Salient Points: Disentangled attention lets the model separately reason about what a word means and where it sits relative to the question versus the candidate answer - useful here since the same words can appear in both correct and incorrect options, so their positional relationship to the question is so important.

Fine-Tuning Strategy: Full fine-tuning of the ~449M-parameter model was avoided in favour of LoRA, which freezes the pretrained backbone and injects small trainable rank-decomposition matrices into the query_proj, key_proj, value_proj, and dense layers of each attention block (r=32, alpha=64, dropout=0.1). This left only 14,223,362 of 449,287,172 parameters trainable (3.17%). Training used the Hugging Face Trainer with AdamW, learning rate 2e-5, a linear warm-up over the first 10% of steps, weight decay 0.01, gradient-norm clipping at 1.0, an effective batch size of 32 (per-device batch 8 × gradient-accumulation 4), and 6 epochs with early stopping (patience 2) on validation F1. A class-weighted cross-entropy loss (weight 4.0 on the minority 'correct' class, matching the 4:1 imbalance) prevented collapse to an 'always incorrect' predictor. FP16/BF16 mixed precision were disabled in favour of stable FP32 training.

Quantization: Not applied in this project.

5.2 Model 2: Custom Deep Learning Model (Text-CNN)

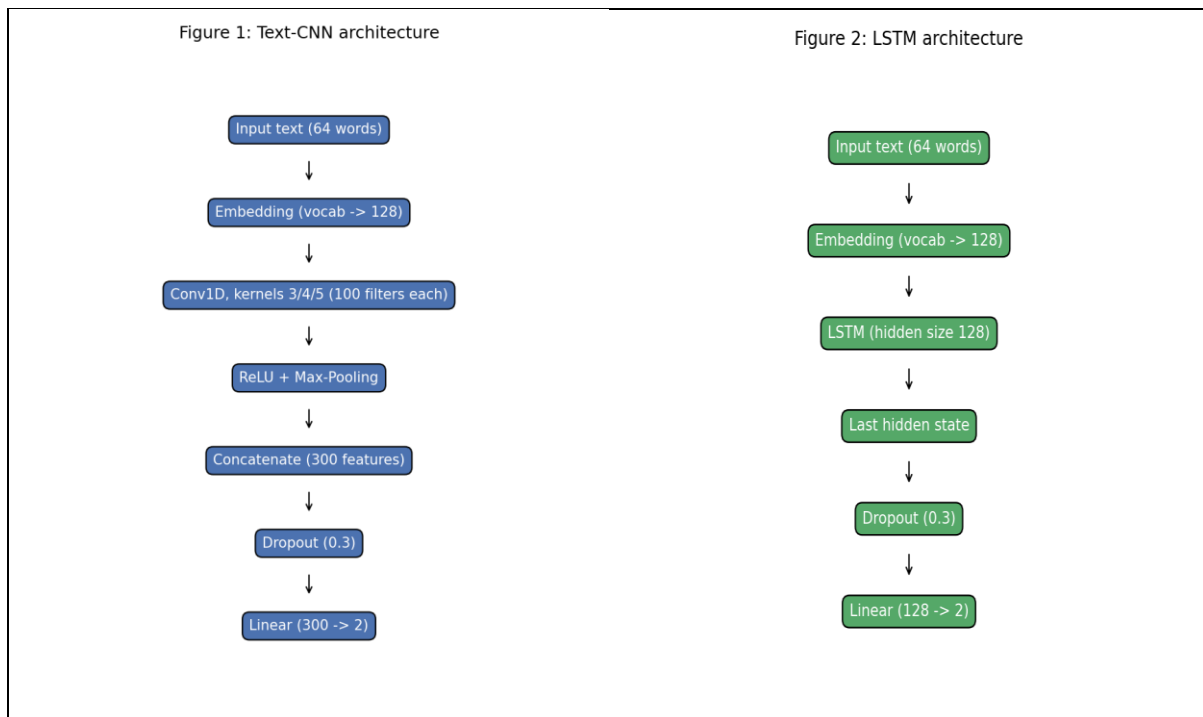
Architecture: A Text-CNN was implemented from scratch in PyTorch: a randomly-initialised embedding layer (DeBERTa's subword vocabulary (~128K tokens), 128-dim, no pretrained weights) feeds three parallel 1D convolutions with kernel sizes 3, 4, and 5 (100 filters each, acting as trigram / 4-gram / 5-gram detectors), each followed by ReLU and max-pooling over the time dimension. The three pooled outputs (300 features total) are concatenated, passed through dropout (0.3), and fed to a linear classifier producing 2 output logits.

Salient Points: The Text-CNN's filters detect local n-gram patterns irrespective of their position in the sentence, which suits this dataset's short phrasing a position-sensitive model would need to separately learn each variant, whereas the CNN's translation pooling generalises across them more readily.

5.3 Model 3: RNN-Based Model (LSTM)

Architecture: A single-layer, unidirectional LSTM was implemented from scratch, sharing the same DeBERTa's subword vocabulary (~128K tokens) / 128-dim embedding layer as Model 2. The LSTM has a 128-dim hidden state and processes tokens left to right; the final hidden state is passed through dropout (0.3) and a linear classifier producing 2 output logits.

Salient Points: Unlike the position-invariant Text-CNN, the LSTM explicitly models sequential dependencies and word order via its recurrent hidden state - word order changes meaning (e.g. "X causes Y" vs "Y causes X").



6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary competition metric is MAP@3, computed by re-assembling the five per-option scores for each question, ranking them, and applying the 1 / 1/2 / 1/3 / 0 scoring rule. During development, per-row Accuracy and F1-score (treating the highest-scored option per question as a single hard prediction) were used alongside a 'predicted-positive-rate' sanity check (the fraction of options predicted 'correct', which should sit near the true base rate of 0.20) to monitor training health at the epoch level and catch failure modes such as class collapse before converting scores into ranked MAP@3 submissions.

6.2 Training Performance

Model 1 (DeBERTa-v3-large + LoRA) trained for 6 epochs; its logged eval/f1 run and eval/loss fell from a high initial value to 0.187 by the final evaluation, with the Trainer's early-stopping/best-checkpoint selection (metric_for_best_model="f1") restoring the strongest checkpoint at the end of training. Models 2 and 3 were trained for 8 epochs each with an explicit per-epoch train/validation loss both

show the expected monotonic decline in training loss, with the Text-CNN's validation loss reaching its minimum at epoch 6 (0.095) before a small bounce, and the LSTM's validation loss declining more smoothly to a minimum of 0.140 at epoch 8.

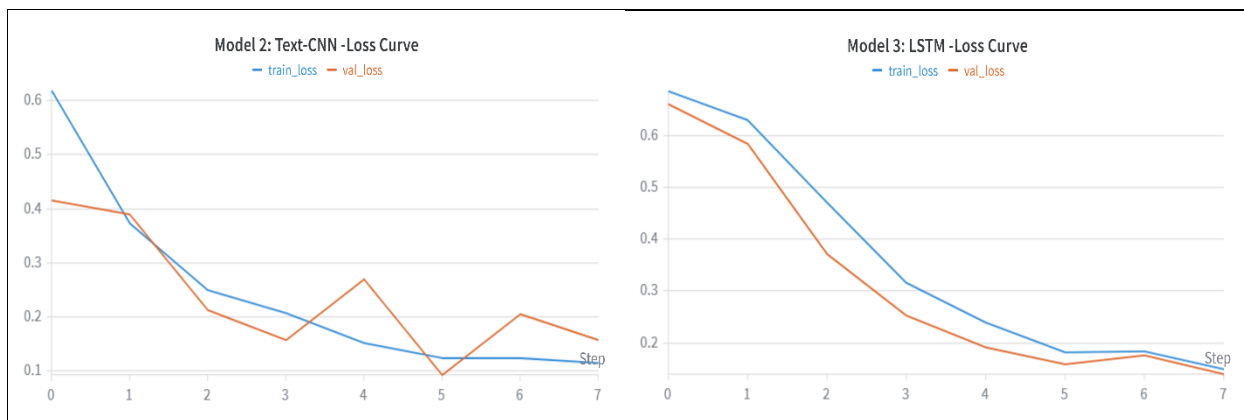


Figure 3: Per-epoch train / validation loss for Model 2 (Text-CNN, left) and Model 3 (LSTM, right)

6.3 Comparative Report

Model	Val. MAP@3	Val. Accuracy	Val. F1
Model 3: LSTM (RNN, from scratch)	0.921	0.956	0.89
Model 1: DeBERTa-v3-large + LoRA (fine-tuned transformer)	0.934	0.956	0.89
Model 2: Text-CNN (custom, from scratch)	0.971	0.98	0.95
Final weighted ensemble (LoRA 0.65 / Text-CNN 0.20 / LSTM 0.15)	0.9933	0.9947	0.9867

Figure 4: Validation MAP@3, Accuracy, and F1 across the three individual models and the final ensemble.

Text-CNN outperformed the fine-tuned transformer on every validation metric, despite having no pretrained language knowledge and a much smaller (2,976-word) vocabulary. Because dataset's short, template-driven phrasing rewards the CNN's translation-invariant local n-gram detectors particularly well, while DeBERTa-v3-large's much larger capacity may be under-utilised on only 8,500 training rows within 6 epochs. The LSTM's sequential modelling did not add value over the CNN's local pattern-matching on this dataset, scoring the lowest of the three individual models. Ensembling all three, using a weighted average that gives the transformer the largest single weight but still lets the two from-scratch models contribute, improved on the best individual model (the Text-CNN) on every metric - evidence that the three architectures, despite differing widely in raw individual performance, learn at least partially complementary error patterns.

6.4 Kaggle Performance

The final weighted ensemble (LoRA 0.65 / Text-CNN 0.20 / LSTM 0.15) was used to score all five options for each of the 2,500 test rows (500 questions) and generate the ranked top-3 submission.csv (score: 0.75020)

7. Conclusion & Future Work

7.1 Key Learnings

- Splitting at the question level (not the row level) was essential to get an honest validation signal, since a row-level split would let different options of the same question leak across the train/validation boundary.
- Systematic, single-variable experimentation is essential: several training failures during this project (e.g. the LoRA rank-64 collapse) occurred specifically when multiple hyperparameters were changed simultaneously, making the root cause hard to isolate. Reverting to a known-good configuration and changing exactly one variable at a time consistently produced interpretable, reproducible results.
- Class-weighted loss was necessary across all three models: the 4:1 (wrong:correct) imbalance from exploding each question into five rows would otherwise bias every model toward a always incorrect predictor.
- Ensembling heterogeneous architectures (a fine-tuned transformer, a CNN, and an RNN) gave a real, reproducible improvement over the strongest single model.

7.2 Challenges Faced

- A HuggingFace datasets formatter bug (an unavailable torchvision.io.VideoReader import) repeatedly interrupted DataLoader-based training/inference; resolved by converting batches to plain PyTorch tensors manually instead of relying on the library's automatic formatting.
- FP16 gradient scaling was incompatible with DeBERTa-v3's disentangled attention, and BF16 proved unreliable on the available T4 GPUs; both were disabled in favour of stable FP32 training.
- Severe class imbalance (4 incorrect : 1 correct per question) required a class-weighted cross-entropy loss (weight 4.0 on the 'correct' class) across all three models to avoid collapse to a trivial predictor.
- Increasing LoRA rank to 64 caused a full training collapse on two separate attempts, showing that added capacity is not always beneficial without careful learning-rate/warm-up tuning.

7.3 Areas for Improvement

- Use k-fold cross-validation at the question level for a less noisy estimate of real-world performance during model selection and hyperparameter search.
- Investigate why DeBERTa-v3-large underperformed the simpler Text-CNN on validation by checking LoRA rank, learning rate, and epoch count - to see whether the transformer's give better result on this dataset.
- Apply light data augmentation (e.g. paraphrasing the question templates identified in Section 3.2) to test whether it narrows the gap between the from-scratch models and the transformer.

8. References

- He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. Proceedings of EMNLP 2014.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
- Hugging Face Model Card: microsoft/deberta-v3-large. huggingface.co/microsoft/deberta-v3-large
- Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP 2020: System Demonstrations. (Hugging Face transformers library)
- Weights & Biases documentation. wandb.ai/site